



מבוא למדעי המחשב מ'ח' (234114 \ 234117)

סמסטר חורף תש"פ

מבחן מסכם מועד ב', 17 במרץ 2020

2	3	4	1	1	
---	---	---	---	---	--

רשום/ה לקורס:

--	--	--	--	--	--	--	--	--	--

מספר סטודנט:

משך המבחן: 3 שעות.

חומר עזר: אין להשתמש בכל חומר עזר.

הנחיות כלליות:

- מלאו את הפרטים בראש דף זה ובדף השער המצורף, בעט בלבד.
- בדקו שיש 22 עמודים (4 שאלות) במבחן, כולל עמוד זה.
- כתבו את התשובות על טופס המבחן בלבד, במקומות המיועדים לכך. שימו לב שהמקום המיועד לתשובה אינו מעיד בהכרח על אורך התשובה הנכונה.
- העמודים הזוגיים בבחינה ריקים. ניתן להשתמש בהם כדפי טיוטה וכן לכתיבת תשובותיכם. סמנו טיוטות באופן ברור על מנת שהן לא תיבדקנה.
- יש לכתוב באופן ברור, נקי ומסודר, ובעט בלבד.
- בכל השאלות, הנכם רשאים להגדיר ולממש פונקציות עזר כרצונכם. לנוחיותכם, אין חשיבות לסדר מימוש הפונקציות בשאלה, ובפרט ניתן לממש פונקציה לאחר השימוש בה.
- אלא אם כן נאמר אחרת בשאלות, אין להשתמש בפונקציות ספריה או בפונקציות שמומשו בכיתה, למעט פונקציות (swap, strlen), הקצאת זיכרון (malloc, free) ופונקציות קלט/פלט. ניתן להשתמש בטיפוס bool המוגדר ב-stdbool.h
- אין להשתמש במשתנים סטטיים וגלובאליים אלא אם נדרשתם לכך מפורשות.
- כשאתם נדרשים לכתוב קוד באילוצי סיבוכיות זמן/מקום נתונים, אם לא תעמדו באילוצים אלה תוכלו לקבל בחזרה מקצת הנקודות אם תחשבו נכון ותצינו את הסיבוכיות שהצלחתם להשיג.
- נוהל "לא יודע": אם תכתבו בצורה ברורה "לא יודע/ת" על שאלה (או סעיף) שבה אתם נדרשים לקודד, תקבלו 20% מהניקוד. דבר זה מומלץ אם אתם יודעים שאתם לא יודעים את התשובה.
- נוסחאות שימושיות:

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \Theta(\log n) \quad \log 1 + \log 2 + \dots + \log n = \Theta(n \log n)$$

$$1^k + 2^k + 3^k + \dots + n^k = \Theta(n^{k+1}) \quad 1 + k + k^2 + k^3 + \dots + k^n = \Theta(k^n), k > 1$$

צוות הקורס 234114/7

מרצים: פרופ' תומר שלומי (מרצה אחראי), ד"ר יוסי ויינשטיין, ד"ר אהרון קופרשטוק, מר איהאב וותד
מתרגלים: עמית ברכה, דן ברקוביץ, גיא ברשצקי, עמית גנץ, עדי גרוס, עמר דהרי, קטרין חדאד, אברנם טל, תומר לנגה, גסוב מזאוי, עמר צברי, דמיטרי רבינוביץ' (מתרגל אחראי), אמאני שחאדה

בהצלחה!

[illegible]



שאלה 1 (25 נקודות):

א. (9 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f1$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן. הניחו שסיבוכיות הזמן של $\text{malloc}(n)$ ו- free היא $\Theta(1)$, וסיבוכיות המקום של $\text{malloc}(n)$ היא $\Theta(n)$.

```
void f1(int n)
{
    int i, j, k = 0;
    for (i = n / 2; i <= n; i++)
    {
        for (j = 2; j <= n; j *= 2)
        {
            k = k + n / 2;
        }
    }
    free(malloc(k));
}
```

סיבוכיות זמן: $\Theta(\quad)$ סיבוכיות מקום: $\Theta(\quad)$

ב. (8 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f2$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
void f2(int n)
{
    for (int i = 1; i < n; i *= 3, n *= 2)
    {
        printf("X");
    }
}
```

סיבוכיות זמן: $\Theta(\quad)$ סיבוכיות מקום: $\Theta(\quad)$

[illegible]



ג. (8 נקודות): חשבו את סיבוכיות הזמן והמקום של הפונקציה $f3$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
void f3(int n)
{
    if (n == 0) return;

    int k = 16;

    for(int i = 0; i * i < k; ++i)
        f3(n / k);

    printf("*");
}
```

סיבוכיות זמן: $\Theta(\text{_____})$ סיבוכיות מקום: $\Theta(\text{_____})$

[illegible]



שאלה 2 (25 נק')

נתון מערך `arr` באורך $n > 3$ שמכיל רצף של איברים עוקבים מסדרה חשבונית. תת-סדרה אחת של הסדרה החשבונית (באורך של איבר אחד לפחות) חסרה.

תזכורת: סדרה חשבונית היא סדרה של מספרים, שבה ההפרש בין כל שני איברים עוקבים הוא קבוע: $a_{n+1} - a_n = d$, כאשר a_n הוא האיבר ה- n -י בסדרה.

ממשו את הפונקציה

```
int FindMissingIndex(int arr[], int n)
```

שמוצאת את האינדקס במערך בו היה אמור להופיע האיבר הראשון בתת-הסדרה החסרה.

דוגמה: עבור המערך $\{22, 24, 26, 32, 34, 36, 38, 40\}$ הפונקציה תחזיר 3, כי זהו האינדקס בו היה אמור להופיע האיבר הבא אחרי המספר 26 בסדרה החשבונית.

דרישות:

- ממשו את הפונקציה בסיבוכיות זמן $O(\log n)$.

אם לא עמדתם בדרישות הסיבוכיות, אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

```
int FindMissingIndex(int arr[], int n)
```

```
{
```

[illegible]

[illegible]

[illegible]



שאלה 3 (25 נקודות) :

ממשו את הפונקציה CountPermutations המקבלת שתי מחרוזות s_1 באורך n ו- s_2 באורך m , ומחזירה את מספר האינדקסים ב- s_1 בהם מתחילה פרמוטציה של מחרוזת s_2 .

```
int CountPermutations(char * s1, char * s2)
```

שימו לב: ניתן להניח כי s_1 ו- s_2 מורכבות מאותיות קטנות באנגלית בלבד.

דוגמה: $s_2 = "aab"$, $s_1 = "abaabababbbaa"$

abaabababbbaa

aba

baa

aab

aba

aba

baa

בדוגמה, פרמוטציות של s_2 מופיעות 6 פעמים בתוך s_1 .

דרישות:

- סיבוכיות זמן $O(n \cdot m)$ וסיבוכיות מקום נוסף $O(1)$.
- פתרון בסיבוכיות זמן $O(n)$ וסיבוכיות מקום נוסף $O(1)$ יקנה בonus של 5 נקודות.

אם לפי חישוביכם לא עמדתם בדרישות הסיבוכיות אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

```
int CountPermutations(char * s1, char * s2){
```

[illegible]

[illegible]

[illegible]



שאלה 4 (25 נקודות) :

נתון מספר שלם n . ממשו פונקציה המחשבת את מספר הביטויים החוקיים המכילים n זוגות סוגריים ' (' ו- ') . ביטוי המכיל סוגריים יקרא **חוקי**, אם בכל רישא (התחלה) שלו מספר הסוגרים (כלומר ה ' (' לא גדול ממספר הפותחים (כלומר ה ' (').

```
int CountParenthesesPairs (int n)
```

דוגמה:

בהינתן $n=3$, הביטויים החוקיים הם אלא ולכן הפונקציה תחזיר את הערך 5.

((()))

(())

(()) ()

() (())

() () ()

הערות:

- יש להשתמש בשיטת backtracking כפי שנלמדה בכיתה.
- בשאלה זו אין דרישות סיבוכיות, אולם כמקובל ב-backtracking יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר (ויש לממש את כולן).

```
int CountParenthesesPairs (int n)
```

```
{
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]